

Description and Technical Details of the Data Preparation Pipeline Experiments

Anonymous Authors¹

1. Clean Crawled Corpus

C4. The Colossal Clean Crawled Corpus (C4) pipeline is composed of a sequence of processing stages applied to raw Common Crawl WET files. As summarized in Table 1, these stages span seven broad categories: (1) URL (which is part of the Common Crawl WET metadata) normalization and canonicalization together with URL-based domain filtering; (2) text-level cleaning using heuristic rules that remove templated, malformed, or low-quality lines; (3) intra-document and corpus-level line deduplication; (4) language identification; (5) bad-word filtering with language-specific rules; (6) optional RealNews/WebText-like filtering based on domain lists; and (7) dataset splitting, serialization, and TFDS integration.

Although the public C4 GitHub repository¹ provides only the processed data, the full construction pipeline is implemented within TensorFlow Datasets (TFDS) in `c4.py`² and `c4_utils.py`³. The `c4.py` module orchestrates Apache Beam pipelines, manages WET file paths, domain lists, split generation, and TFRecord serialization. In contrast, `c4_utils.py` contains most of the document-level cleaning logic: WET parsing, URL normalization, line-level heuristic cleaning, duplicate-line removal, language detection, bad-word filtering, and RealNews/WebText-like rules.

Our reconstruction focuses on the subset of C4 components that perform *natural-language text cleaning* and operate purely at the per-document level. We directly call the original C4 implementations of `clean_page` (via `get_clean_page_fn`), `is_valid_length`, `detect_english`, and the English branch of `get_badwords_filter_fn`. These functions de-

pend only on the textual content of a `PageFeatures` object, making them fully compatible with plain-text inputs.

Several of these components rely on information or infrastructure intrinsic to the original C4 construction. In particular, metadata-dependent operations (e.g., URL canonicalization, RealNews/WebText filtering), WET parsing, and global deduplication are executed within Apache Beam pipelines using distributed primitives such as `CoGroupByKey`, `CombinePerKey`, and `Top.PerKey`. These stages cannot operate outside the Beam execution model or without full Common Crawl metadata. We discard all components that inherently require Common Crawl metadata, URL structure, domain lists, or Beam-wide coordination. These include WET/WARC parsing, URL and timestamp extraction, URL-based canonicalization and deduplication, RealNews/WebText-like domain filtering, CLD3 multilingual detection, paragraph-level heuristics for Multilingual C4 (mC4), Beam-based canonical-document deduplication, and TFDS dataset construction. Since these steps are either irrelevant to natural-language text cleaning or cannot be reproduced without the underlying metadata, they fall outside the scope of our reconstruction. For C4 components whose behavior is relevant to natural-language text cleaning but whose implementations depend on corpus-level Beam state—most notably intra-document and global line-level deduplication—we reconstruct analogous functionality using local, single-machine implementations. Our versions reproduce C4’s MD5-based line hashing logic but replace Beam’s distributed `CoGroupByKey/CombinePerKey` operations with lightweight in-memory data structures. This preserves the functional effect of C4’s deduplication rules while remaining compatible with our setting.

A complete mapping between the full C4 pipeline and our system is provided in Table 1.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

¹<https://github.com/allenai/c4-documentation>

²https://github.com/tensorflow/datasets/blob/master/tensorflow_datasets/text/c4.py

³https://github.com/tensorflow/datasets/blob/master/tensorflow_datasets/text/c4_utils.py

Data Provenance Auditing of Fine-Tuned Large Language Models with a Text-Preserving Technique

Table 1. Comparison between the full C4 pipeline functionality and our implementation. We distinguish components that are not included, components that are directly reused from `c4_utils.py`, and components that are reimplemented with equivalent or analogous logic in a non-distributed setting.

Category	C4 component (key functions / classes)	Status in our system	Notes
URL-level processing and domain filtering		Not included	Not applicable since the dataset is not scrapped from the web
Text-level cleaning and quality filtering			
Length filtering	<code>is_valid_length</code>	Directly called	We wrap <code>is_valid_length</code> and apply the same 190k-character cutoff.
Line-level heuristic cleaning (English)	<code>clean_page</code> (via <code>get_clean_page_fn</code>)	Directly called	We call <code>get_clean_page_fn</code> on a <code>PageFeatures</code> constructed from our text and reuse C4’s line filtering rules.
Paragraph-based filtering (mC4)	<code>paragraph_filter</code>	Not included	Paragraph heuristics are designed for multilingual C4; we focus only on English C4-style cleaning.
Deduplication			
Intra-document duplicate line removal	Local component of <code>remove_duplicate_text</code> (per-document line hashing)	Reimplemented (Beam-free)	We implement intra-document line deduplication using MD5-based hashing over lines (function <code>dedupe_lines_c4_style</code>), following the same idea but without Beam.
Global line-level deduplication with canonical URLs	Full <code>remove_duplicate_text</code> (Beam pipeline with <code>Top.PerKey</code> , <code>CoGroupByKey</code>)	Reimplemented (Beam-free)	We implement the same MD5-based line hashing and filtering logic used in C4, but execute it locally rather than through Beam’s distributed <code>CoGroupByKey/CombinePerKey</code> operations. For our setting, the functional behavior is equivalent.
Language identification			
English language detection	<code>detect_english</code> (langdetect with seeded <code>DetectorFactory</code>)	Directly called	We wrap <code>detect_english</code> and accept a page as English iff the C4 detector yields it, ensuring identical behavior.
Multilingual language detection (CLD3)	<code>detect_languages</code> (CLD3 over many languages)	Not included	Our system targets English-only data and does not construct mC4-style multilingual splits.
Bad-word filtering and content safety			
Bad-word filtering (English)	English branch of <code>get_badwords_filter_fn</code> with LDNOOBW list	Directly called	We build an English bad-word dictionary from the same LDNOOBW source and use <code>get_badwords_filter_fn</code> on <code>PageFeatures</code> with <code>language = "en"</code> .
Bad-word filtering (other languages)	Non-English branches of <code>get_badwords_filter_fn</code>	Not included	We do not filter non-English content because our pipeline is English-only.
Dataset splitting and TFDS integration		Not Included	No TFDS or TFRecord shards output

2. CCNet

CCNet. The original CCNet pipeline⁴ is designed for large-scale CommonCrawl processing, and therefore includes numerous components that require WARC/WET ingestion, sharded hash-file infrastructure, JSONQL pipeline orchestration, and distributed execution via the Executor framework. Because our dataset does not consist in raw web crawl segments, many CCNet components cannot be directly reused. In particular, the official pipeline assumes (i) access to raw CommonCrawl segments; (ii) precomputed cross-shard dedup hash bins; and (iii) documents represented as `raw_content` streams produced by `CCShardReader`. None of these features exists in our setting, preventing the direct use of `mine()`, `hashes()`, or `regroup()` as end-to-end data-cleaning functions.

We reconstruct the CCNet document-processing pipeline so that each cleaning stage—normalization-based deduplication, fastText-based language identification, SentencePiece tokenization, KenLM perplexity scoring, and perplexity bucketing—can operate directly on plain-text JSONL documents. As shown in Table 2, several components are *directly reused* from the official implementation, including the normalization function `normalize_for_dedup`, the fastText-based language identification module `split_by_lang.Classifier`, the SentencePiece tokenizer `MultiSentencePiece`, the KenLM perplexity scorer `DocLM`, and the perplexity bucketing logic implemented in `PerplexityBucket`. These modules operate purely on Python dictionaries and therefore can be applied unchanged to JSONL documents.

Other components must be *reimplemented* because they are part of the distributed ingest pipeline. For example, global deduplication in the official system relies on shard-level hash bins generated by `HashesCollector`, but our data lacks shards or CC metadata. To replicate this functionality, we hash normalized document text with an in-memory SHA-1 set. Similarly, we manually recreate tokenized-field cleanup, which in the official system is handled by `DropKeys` within the JSONQL execution engine. The cutoff file `cutoff.csv` is also regenerated from our own corpus because Wikipedia-trained cutoffs depend on the distribution of large CC-scale data.

Finally, several CCNet components are omitted as they are specific to CommonCrawl processing, such as WARC/WET ingestion, segment-based routing, sharded regrouping, metadata fetching, language whitelisting/blacklisting, and distributed execution. These features are unnecessary for our evaluation of CCNet-style document cleaning.

⁴https://github.com/facebookresearch/cc_net/blob/main/cc_net/mine.py

3. The Pile

The Pile. The Pile data-cleaning pipeline combines the lightweight normalization utilities implemented in its public repository with several additional preprocessing stages described only in (Gao et al., 2020). In the open-source codebase⁵, the only text-cleaning functions actually provided are `strip_markdown_colons` and `remove_advertisement`, applied exclusively to the PubMed Central and OpenWebText2 components, respectively. No general-purpose, dataset-independent cleaning pipeline is exposed. Moreover, we did not find any implementation of the dataset-specific preprocessing steps discussed in the paper—including `jusText`-based boilerplate removal and `pycld2`-based language filtering for Pile-CC, Unicode repair via `ftfy` for BookCorpus2, or MinHashLSH-based global deduplication (Appendix C–D in the Pile paper (Gao et al., 2020))—in the public repository.

To apply Pile-style preprocessing, we reconstruct a functional cleaning pipeline by combining the utilities that do exist in the open-source codebase with reimplementations of all paper-only preprocessing steps. Specifically, we incorporate `jusText`-based boilerplate removal (Pile-CC), Unicode repair via `ftfy` (BookCorpus2), and the English-only filtering step described for Pile-CC, which we implement using the `pycld2` language detector. Furthermore, whereas the original Pile applies different cleaning operators to different subdatasets, our pipeline applies all available cleaning steps uniformly across all inputs, resulting in a stricter and more consistent preprocessing procedure.

A comprehensive, component-level comparison of the full the Pile data-processing pipeline—including both the officially implemented modules in the public repository and the additional preprocessing stages documented only in the original paper—together with the corresponding components reproduced in our reimplementations, is presented in Table 3.

⁵<https://github.com/EleutherAI/the-pile>

Data Provenance Auditing of Fine-Tuned Large Language Models with a Text-Preserving Technique

Table 2. Comparison between the full CCNet pipeline and our implementation. We distinguish components not included, components directly reused from CCNet, and components reimplemented locally with equivalent logic.

Category	CCNet component (key functions / classes)	Status in our system	Notes
Deduplication			
Normalization for dedup	<code>text_normalizer.normalize_for_dedup</code>	Directly called	We call CCNet’s exact normalization function for hashing, ensuring bit-for-bit consistency.
Precomputing shard-level hash files	<code>dedup.HashesCollector</code> , <code>hashes(conf)</code> generating <code>*.bin</code> files	Not included	Official dedup uses CC-scale external hash files; our system uses an in-memory hash set.
Global dedup across shards	<code>dedup.DuplicatesRemover</code> (operates on CC shards with hash files)	Reimplemented	We reimplement, locally instead of distributed, functionally equivalent document-level dedup using SHA1 over normalized text.
Line-level dedup inside document	Handled implicitly inside CCNet’s pipeline (not exposed as public API)	Not Included	Our documents contain one text unit per JSON entry; no multi-line CC-style dedup required.
Language Identification			
FastText LID classifier	<code>split_by_lang.Classifier</code>	Directly called	We directly apply the same classifier to each JSONL document, producing identical labels and scores.
Pre/post-dedup LID (CC-specific)	<code>lid.before_dedup</code> , <code>lid.after_dedup</code>	Not included	These are CC-scale diagnostics; not needed for our dataset.
Language whitelist/blacklist filtering	<code>keep_lang</code> , filters based on config	Not included	We keep all languages; no whitelist/blacklist logic is needed.
SentencePiece tokenization and LM scoring			
SentencePiece tokenization	<code>perplexity.MultiSentencePiece</code>	Directly called	We run MultiSentencePiece exactly as in CCNet.
KenLM language modeling	<code>perplexity.DocLM</code>	Directly called	We reuse CCNet’s LM scoring for perplexity computation.
Drop tokenized intermediate field	<code>perplexity.DropKeys</code>	Reimplemented	We manually delete the tokenized field to match CCNet’s cleanup behavior.
Perplexity bucketing			
Cutoff percentile loading	<code>cutoff.csv</code> + <code>PerplexityBucket</code>	Directly called	We call the same bucketing function; <code>cutoff.csv</code> is regenerated from our dataset.
Official CCNet cutoff values	Downloaded from <code>data/cutoff.csv</code> trained on Wikipedia LMs	Recomputed locally	We compute percentile cutoffs from our own distribution instead of CC global thresholds.
Filtering and routing			
Length filtering on raw documents	<code>min_len</code> (filtering raw content < 300 chars)	Not included	Not applicable because our documents are already curated; no short-page filtering.
Bucket-based retention	<code>keep_bucket</code> (retain head/middle/tail)	Reimplemented (optional)	Our system computes buckets, and (optionally) checks if the watermarked text would be filtered by the quality bucket.
Metadata fetching	<code>minify.MetadataFetcher</code> (CCNet 1.0 reproduction mode)	Not included	Requires CC-provided metadata; irrelevant for our dataset.
Field minification	<code>minify.Minifier</code>	Not included	Our JSON does not contain CC metadata fields needing minification.

Data Provenance Auditing of Fine-Tuned Large Language Models with a Text-Preserving Technique

Category	CCNet component (key functions / classes)	Status in our system	Notes
Dataset splitting and redistribution		Not included	We do not split the dataset in shards

Table 3. Comparison between the full The Pile data-processing pipeline (as described in the original paper), the corresponding components implemented in the official GitHub repository, and the components included in our cleaning pipeline. We distinguish functionality that is not included, functionality directly reused from the open-source code, and functionality reimplemented with same logic.

Category	The Pile component (paper & code)	Status in our system	Notes
Text-level cleaning and normalization			
Markdown directive removal	<code>strip_markdown_colons</code> (official code, paper: Appendix C.2 Pubmed Central dataset)	Directly called (but applied globally)	We call the same regex-based markdown removal function. The Pile applies this only to Pubmed Central dataset; our pipeline applies it to all documents.
Advertisement / boilerplate removal	<code>remove_advertisement</code> (official code, OWT2 dataset)	Directly called (but applied globally)	The Pile applies this only to OpenWebText2; our pipeline applies it to all documents.
Math dataset chunking	<code>chunk_at_even_lines</code> (official code, DMMathDataset)	Not included	Our data is not hierarchical math reasoning content; chunking is unnecessary.
GitHub large-file filtering	GitHubDataset: skip files >100k chars (official code, GithubDataset)	Not included	We do not filter by maximum length other than our own minimum-length threshold.
HTML tag removal and boilerplate stripping	Paper: “higher-quality extraction with jusText” (Appendix C; used for web-scraped datasets such as Pile-CC)	Reimplemented (applied globally)	Our <code>clean_html</code> uses <code>jusText</code> for boilerplate detection and removal.
Unicode normalization / fix-up	Paper: explicitly applies <code>ftfy</code> only to BookCorpus2 (Appendix C)	Reimplemented (applied globally)	We use the <code>ftfy</code> library to repair mojibake and normalize punctuation and apply to all document.
Global / document-level deduplication			
Document-level MinHashLSH deduplication (OpenWebText2 & Pile-CC)	Paper: Appendix D.2 and Appendix C.1.4/C.4 (MinHashLSH over OpenWebText2 and the Common Crawl subset Pile-CC)	Reimplemented (applied globally)	We implement a simplified single-machine MinHashLSH deduplication using the same 13-gram shingling approach described in the paper.
Language identification & statistics			
English Language filtering	Paper: Appendix C (filtering Pile-CC dataset with <code>pycld2</code>)	Reimplemented (applied globally)	The Pile applies explicit English-language filtering only to the Pile-CC subset in the paper; the official repository does not include any language-filtering operator. We reintroduce this paper-described component and apply it universally using <code>pycld2</code> .
Dataset mixing and size control			
Dataset-weighted interleaving	<code>PileReplication</code> (official code)	Not included	We do not replicate Pile’s weighted multi-dataset sampling.
Random sub-sampling to a fixed byte budget	<code>LimitedDataset</code> (official code)	Not included	Our workflow processes all user-provided samples.

4. RedPajama

RedPajama. The RedPajama data-processing workflow⁶ is built around a multi-stage, distributed system that operates on CCNet-preprocessed Common Crawl segments. As outlined in Table 4, the full pipeline incorporates: (1) construction of document objects followed by evaluation of an extensive suite of quality signals—including content heuristics, repetition indicators, natural-language likelihood metrics, line-level structure checks, and additional classifier-based and DSIR-based signals; (2) generation of MinHash signatures via normalization, token n -gram generation, and permutation-based hashing; (3) fuzzy deduplication through LSH banding, bucket aggregation, and graph clustering; (4) distributed writing of `*.signals.json.gz` and `*.minhash.parquet` files

Many of these stages are inseparable from the assumptions and infrastructure of the official RedPajama system. The core worker implementation expects CCNet-formatted records containing fields such as `raw_content`, `perplexity`, `bucket`, `nlines`, `language_score`, and `source_domain`, and performs snapshot-based Amazon S3 I/O to read CCNet shards and emit outputs. Additionally, several quality-signal modules rely on large auxiliary artifacts—including bad-word lexicons, URL blocklists, fastText classifier checkpoints, and DSIR feature arrays—that must be produced from external corpora (e.g., Wikipedia, Books, OpenWebText) and are not bundled with the public release. Moreover, the implementation of MinHash and signal computation is deeply tied to the worker execution loop and cannot be invoked independently of the distributed CCNet reader. Consequently, the full RedPajama pipeline cannot be executed directly on arbitrary plain-text inputs.

In our setting, the input consists of raw texts rather than CCNet `.json.gz` documents. As a result, components that rely on CCNet metadata, external classifier or DSIR artifacts, or the multi-process orchestration layer cannot be reused as-is. This includes the CCNet-mode `Reader`, the classifier and DSIR signal modules, and the URI-based processing logic within the worker implementation.

Our reconstruction therefore concentrates on the subset of RedPajama components that provide *document-level text cleaning and near-duplicate detection* and that remain functional for plain-text inputs. We directly adopt the official `Document` class, including its normalization procedures, token n -gram extraction, and hash-feature precomputation. We likewise reuse the built-in quality-signal modules for content quality, repetition, natural-language identification, and line-level heuristics, as well as the official `MinHash`

implementation and the associated search procedure for banding parameters. These components depend solely on the document text and are fully compatible with our usage.

For parts of the RedPajama system whose high-level behavior is necessary but whose implementation is intertwined with worker-level execution—in particular, LSH bucket construction and clustering—we provide local reconstructions. Our implementation mirrors the structure of RedPajama’s fuzzy deduplication: it generates banded MinHash signatures, performs bucket grouping using the same keying rules, constructs edges by linking each item to the minimum `id_int` in its bucket, and applies Networkit connected-component analysis. We replace distributed shard writers with a single local Parquet writer while preserving the schema used by RedPajama.

Conversely, we omit those parts of the original system that fundamentally rely on CCNet metadata, fastText classifier models, DSIR feature matrices, worker orchestration, or distributed I/O. This includes all CCNet-derived quality metrics, classifier-based signals, DSIR-based signals, snapshot-aware routing, and the production of shard-level `*.signals.json.gz` files. These stages are either infeasible to reproduce without the corresponding metadata and artifacts or are irrelevant to the document-level text-cleaning tasks that our study focuses on. A full correspondence between the original RedPajama workflow and our reconstruction is shown in Table 4.

⁶<https://github.com/togethercomputer/RedPajama-Data>

Data Provenance Auditing of Fine-Tuned Large Language Models with a Text-Preserving Technique

Table 4. Comparison between the full RedPajama data-processing pipeline and our implementation. Components are categorized as directly reused, omitted, or added as extra analysis stages.

Pipeline Stage	RedPajama Component (key functions / scripts)	Status in our system	Notes
Document construction and quality signal computation (Worker._process_record)			
Document preprocessing (ngrams, hash features)	core.document.Document	Directly called	We construct Document objects with <code>precompute_ngrams=True</code> and <code>precompute_hash_features=True</code> , matching the official Worker.
Content quality signals (e.g., stopword ratios, symbol density, bad URLs/words)	register_content_callable	Directly called	We call RedPajama’s content-signal registration for <code>lang="en"</code> and apply all available content heuristics.
Repetition signals (repeated spans, boilerplate)	register_repetitions_callable	Directly called	Repetition-related signal functions are reused as in the official Worker.
Natural-language vs non-natural-language signals	register_natural_language_callable	Directly called	We include RedPajama’s natural-language detection signals in our quality-signal set.
Line-level heuristic signals	register_lines_callable	Directly called	Line-level quality heuristics are registered and computed in the same way as in the Worker.
FastText-based ML classifiers (palm, wikiref, wikipedia)	register_classifier_callable	Not included	Requires large <code>fastText *.bin</code> models trained from full Wikipedia/OWT/Books/CCNet; these are not downloaded in our environment and are not essential for testing cleaning heuristics.
DSIR importance weights (per-domain mixture weights)	register_importance_weights_callable	Not included	Depends on DSIR artifacts (<code>*.counts.npy</code> , <code>*.lambda.npy</code>) computed from large external datasets; we omit them because they are not required for our cleaning-focused experiments.
CCNet-derived quality signals (length, nlines, perplexity, bucket)	Inline in Worker._process_record as <code>ccnet_</code> fields	Not included	Our input does not contain CCNet statistics, so the <code>ccnet_length</code> , <code>ccnet_perplexity</code> , etc. fields are absent.
MinHash signature computation (Worker._process_record)			
Normalization (lowercasing, whitespace compression, Unicode NFD)	utilities.text.normalization.normalize (used by Document)	Directly called	We use the same normalization as the official Worker before MinHash tokenization.
Token n-gram construction (3-grams over normalized words)	utilities.text.ngrams.form_ngrams (inside MinHash)	Directly called	Our tokens for MinHash are generated exactly as in RedPajama.
MinHash permutation setup (Mersenne prime, 32-bit hashes)	dedupe.minhash.MinHash constructor	Directly called	We instantiate the official MinHash with the same parameters (<code>similarity.thresholds</code> , <code>ngram.size</code> , <code>num.permutations</code> , <code>seed</code>).
Banded MinHash signatures for each similarity threshold	MinHash.compute_banded_signatures	Directly called	We compute banded signatures with the same method and store them as lists of byte strings per threshold.

Data Provenance Auditing of Fine-Tuned Large Language Models with a Text-Preserving Technique

Pipeline Stage	RedPajama Component (key functions / scripts)	Status in our system	Notes
MinHash Parquet schema (per-shard)	Worker's <code>_minhash_schema</code> (<code>shard_id</code> , <code>id</code> , <code>id_int</code> , <code>signature_simX</code>)	Reimplemented (same schema)	We build a Parquet file with the same logical schema using Polars instead of <code>ParquetBatchWriter</code> .
LSH-based fuzzy deduplication (<code>run_lsh.py</code>)			
Optimal (bands, rows) search for a given similarity	<code>dedupe.utils</code> <code>.optimal_param</code>	Directly called	We reuse exactly the same band/row selection procedure as RedPajama.
Band expansion and bucket grouping over MinHash signatures	<code>run_lsh.py</code> (Polars / PyArrow group-by logic)	Reimplemented (same semantics)	Our implementation follows the official steps: explode bands, group by (<code>signature_simX</code> , <code>band</code>), keep buckets with more than one member.
Graph construction from buckets	<code>networkit.Graph</code> usage in <code>run_lsh.py</code>	Reimplemented (same logic)	We connect each node in a bucket to the minimum <code>id_int</code> in that bucket, matching RedPajama's edge-construction rule.
Connected component calculation (clusters of near-duplicates)	<code>networkit.components</code> <code>.ConnectedComponents</code>	Directly called	We invoke the same Networkit connected components algorithm as in <code>run_lsh.py</code> .
Cluster representative selection	Minimum <code>id_int</code> per connected component	Reimplemented	We define the representative of each cluster as the smallest <code>id_int</code> , as in the official LSH deduper.
Output generation in <code>rp_signals</code> (<code>Worker._process_uri</code>)			
Writing <code>*.signals.json.gz</code> (metadata + all quality signals)	<code>utilities.io.Writer</code> with <code>RP_SIGNAL_SCHEMA</code>	Not included	The official pipeline writes shard-level <code>.signals.json.gz</code> files; we instead attach RP quality signals directly into each JSONL record for analysis.
Writing <code>*.minhash.parquet</code> (per-shard signatures)	<code>ParquetBatchWriter</code>	Reimplemented	We write a single <code>minhash.parquet</code> with the same columns but via Polars, not the streaming batch writer.

5. Dolma

Dolma. Dolma⁷ is designed as a strictly non-destructive data processing system. The source code in `data_types.py` makes this design explicit by fully separating raw document content from the tagging information produced by downstream components. Classes such as `InputSpec` and `Document` store only the original fields (`id`, `source`, `version`, and `text`), and all transformation methods (`from_spec`, `to_spec`, `from_json`, `to_json`) always pass the text through unchanged. No method in these classes performs any form of rewriting, filtering, normalization, or alteration of the input string.

Tagging results are represented independently through span-level structures. The types `TaggerOutputValueType`, `TaggerOutputType`, and `TaggerOutputDictType` encode tagger outputs as lists of tuples (`start`, `end`, `score`) that point back to offsets in the immutable text. The `Span` class wraps these tuples and provides utilities for reading from the document (`select` and `mention`), but it never mutates the underlying content. Similarly, `OutputSpec` and `DocResult` aggregate multiple tag sets while always reusing the untouched original document.

Because the module provides no mechanism to modify text and expresses all tagger outputs as external attribute dictionaries or span lists, Dolma ensures that the input data remains intact. The system therefore acts purely as an annotator: it preserves the exact original document while layering structured metadata on top of it, enabling reproducible, model-independent downstream processing. Therefore we did not conduct any implementation on the Dolma pipeline.

FineWeb. The FineWeb 2 pipeline was designed for large-scale processing of Common Crawl data and includes several stages that assume direct access to web documents in WARC format. As outlined in Table 5, the official workflow begins with WARC ingestion, URL filtering, and HTML extraction, proceeds through GlotLID-based language identification and per-language score thresholding, and then performs MinHash deduplication, a set of language-specific quality filters, and a final round of text normalization and PII removal. Much of this machinery depends on metadata available only in raw web crawls—URLs, timestamps, HTTP headers—or on Datatrove’s distributed Slurm executor.

The public FineWeb 2 repository⁸ provides all configuration files and the underlying code, but offers no simple API for applying the cleaning pipeline to arbitrary text. Instead, the

⁷<https://github.com/allenai/dolma/blob/main/python/dolma/core/taggers.py> and https://github.com/allenai/dolma/blob/main/python/dolma/core/data_types.py

⁸<https://github.com/huggingface/fineweb-2>

implementation is structured around Datatrove’s distributed execution model and assumes input data drawn directly from S3-hosted Common Crawl dumps. Components such as `WarcReader`, `URLFilter`, and `Trafilatura` require HTML content and WARC metadata, while GlotLID-based routing depends on intermediate probability distributions and YAML thresholds that FineWeb 2 supplies only for non-English languages. In particular, no `eng_Latn.yml` file is provided; the English thresholds are defined implicitly through the earlier FineWeb-English stage, which prevents us from reusing the language-score filtering logic as-is.

Because our dataset consists of plain natural-language text rather than raw web documents, many early stages of FineWeb 2 do not apply to our setting. We therefore focus on reconstructing the parts of the pipeline that perform genuine text cleaning and can operate on standalone documents. For language identification, we adopt the approach used in FineWeb-English and apply `LanguageFilter` with the FT-176 backend, using a threshold aligned with the structure of the YAML configurations used for other languages.

For deduplication, we directly invoke all four MinHash components from the official pipeline—`MinhashDedupSignature`, `MinhashDedupBuckets`, `MinhashDedupCluster`, and `MinhashDedupFilter`—with the same hashing parameters (xxhash-64, 14 buckets, 8 hashes, 5-grams). The only change is that we execute these stages sequentially on a single machine instead of through Slurm-based parallelization. We likewise apply the full set of FineWeb 2 quality filters without modification: `GopherRepetitionFilter` for surface-level repetition, `FineWebQualityFilter` for structural cues, and `GopherQualityFilter` for lexical quality. These filters are configured either from the official YAML files or from fallback defaults that follow the same format.

Finally, we preserve FineWeb 2’s text normalization stage. All surviving documents pass through the same formatter sequence—`FTFYFormatter`, `PIIFormatter`, and `SymbolLinesFormatter`—ensuring consistency with the original workflow.

In contrast, we omit components that rely fundamentally on web metadata or distributed infrastructure. These include WARC ingestion, HTML extraction, URL-based filtering, GlotLID-based multilingual routing, Slurm sharding, and the writing of per-language shard files. Such steps are impossible to reproduce faithfully without the underlying web data and do not contribute to text-level cleaning in our scenario. Table 5 provides a detailed mapping of which parts of the FineWeb 2 pipeline are directly reused, which are reconstructed, and which are excluded.

Data Provenance Auditing of Fine-Tuned Large Language Models with a Text-Preserving Technique

Table 5. Comparison between the FineWeb 2 data processing pipeline and our implementation. We group components by the six major stages described in the official pipeline.

Stage	Official FineWeb 2 component	Status in our system	Notes
Raw Web Extraction (FineWeb English pipeline)		Not included	Inputs are raw texts
GlottLID Language Identification		Not included	All texts are in english
Per-Language Language-Score Filtering			
Score thresholding using GlottLID scores	LambdaFilter (above_lang_threshold)	Reimplemented	We did not find an English (eng_Latn) YAML config in the the FineWeb 2 repository. Following the FineWeb-English stage, we apply LanguageFilter (backend="ft176") with a manually aligned threshold.
Per-language YAML config loading	configs/\${lang_script}.yaml	Directly used (fallback for eng_Latn)	We adopt the same config structure, using fallback defaults for English.
MinHash Deduplication (4 stages)			
Signature computation (stage 1)	MinhashDedupSignature	Directly used	Same config: 64-bit xxhash, 14 buckets, 8 hashes, 5-grams.
Bucketization (stage 2)	MinhashDedupBuckets under Slurm	Reimplemented (single-machine)	Identical logic; sequentially loops over bucket ranks.
Clustering (stage 3)	MinhashDedupCluster	Directly used	Single-machine call instead of Slurm.
Duplicate filtering (stage 4)	MinhashDedupFilter	Directly used	Extra exclusion writer added for analysis.
Full Slurm orchestration	SlurmPipelineExecutor	Not included	We run stages manually without distributed compute.
Per-Language Quality Filtering			
Gopher repetition filter	GopherRepetitionFilter	Directly used	Config normalized to official tuple format for top_n_grams/dup_n_grams.
FineWeb quality filter	FineWebQualityFilter	Directly used	Same parameters: short_line_thr=999, char_duplicates_ratio=0.1.
Gopher quality filter	GopherQualityFilter	Directly used	Uses YAML or fallback stop-words + thresholds.
Distributed execution with per-filter exclusion logs	Slurm pipeline writing to removed/	Reimplemented (local loop)	We record the first failing filter in reason_map.
Final Touches (Text Normalization)			
Text encoding fix	FTFYFormatter	Directly used	Same ordering: FTFY → PII → SymbolLines.
PII removal	PIIFormatter	Directly used	Same as official FineWeb 2.
Artifact cleanup (tables etc.)	SymbolLinesFormatter ([" "])	Directly used	Matches official configuration.
Final JSONL output	JsonlWriter (final_touches)	Simplified	We store cleaned text in memory rather than sharded JSONL.

References

Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., Presser, S., and Leahy, C. The pile: An 800gb dataset of diverse text for language modeling, 2020. URL <https://arxiv.org/abs/2101.00027>.